

How To: Code-Review

1. Wie funktioniert ein Code Review

Ein Code Review ist ein Prozess, bei dem jemand anderes als der Autor des Codes diesen nach bestimmten Kriterien überprüft.

Ablauf eines Code Reviews

Üblicherweise arbeiten mehrere Entwickler gemeinsam an einer Code Basis, die Schritt für Schritt um Funktionen erweitert wird. Daher wird eigentlich nie ein gesamtes Projekt komplett reviewt, sondern immer nur die Änderung, die ein einzelner Entwickler an der Code Basis vorgenommen hat.

Der Ablauf eines **Remote Code Reviews** ist im Idealfall folgendermaßen:

1. Entwickler A (der Autor) führt Änderungen am Code durch, z.B. um ein Feature zu ergänzen oder einen Bug zu beheben.
2. Der Autor pusht seine Änderungen über ein Versionskontrollsystem (z.B. Git) auf einen Branch des Projektes (d.h. die "Haupt"-Code-Basis wird nicht verändert, sondern es wird eine Variante des Codes erstellt. Hier gibt es allerdings noch andere Herangehensweisen).
3. Der Autor weist Entwickler B als Code Reviewer zu. Entwickler B (der Code Reviewer) kann fest von der Projektleitung vorgegeben sein, oder er wurde vom Autor selbst ausgewählt. Entwickler B ist häufig Teil desselben Projekts, muss es aber nicht sein.
4. Der Code Reviewer reviewt den Code nach bestimmten Kriterien. Der Code Reviewer teilt seine Kritik und konkrete Änderungsvorschläge mit dem Autor, meistens über ein Code Review Tool wie GitHub.
5. Der Autor führt die geforderten Änderungen am Code durch oder verteidigt seinen Code und erklärt, weshalb er nicht geändert werden sollte.
6. Schritte 4 und 5 werden so lange wiederholt, bis sich beide Parteien einig sind (in extremen Fällen wird ein zweites Code Review von einem weiteren Entwickler angefordert).
7. Die Code-Änderungen werden in die "Haupt"-Code-Basis übernommen.

Dieser Ablauf kann sich durchaus über Tage oder Wochen hinziehen.

Eine andere Form sind **In-Person-Code Reviews**. Diese laufen eher so ab:

1. Der Code Reviewer liest sich unter Anwesenheit des Autors die Code-Änderungen durch. **Der Autor erklärt seinen Code nicht, er ist still.**
2. Der Code Reviewer stellt dem Autor bei Bedarf Fragen, die dieser präzise beantwortet, ohne abzuschweifen.
3. Der Code Reviewer macht dem Autor Änderungsvorschläge. Der Autor hat nun die Chance, seinen Code an dieser Stelle konkret zu verteidigen, oder die Änderungen einzubauen.
4. Das Code Review endet, wenn der Code Reviewer keine Gesprächsthemen mehr hat.

Wichtig bei dieser Form des Code-Reviews ist, dass der Reviewer sich den Code vom Autor nicht erklären lässt, da ein späterer Entwickler diese Möglichkeit auch nicht hat.

Code Review-Kriterien

Die Kriterien für ein Code Review sind meistens nicht exakt vorgegeben, umfassen aber typischerweise folgende Aspekte:

- Funktion:
 - **Setzt der Code die geforderte Funktion um?**
 - Werden Randfälle behandelt?
 - **Gibt es logische Fehler oder Bugs?**
- Software Design:
 - **Fügt sich der Code in das allgemeine Software Design der Code-Basis ein? Falls es Design-Vorgaben gibt, entspricht das Code-Design diesen Vorgaben?**
 - **Ist der Code wiederverwendbar? (Möglichst modularer Aufbau des Codes in einzelnen Methoden)**
 - **Vermeidet der Code Redundanzen? (Möglichst wenig Code, der genau das gleiche tut)**
 - Gibt es überflüssige Abhängigkeiten? (z.B. unnötige Library-Imports, Verwendung von unnötig großen Frameworks für kleine single-use Aufgaben)
- Komplexität und Lesbarkeit:
 - Lässt sich die Funktionsweise des Codes schnell und einfach nachvollziehen?
 - **Könnte man den Code vereinfachen?**
 - Sind komplizierte Algorithmen nachvollziehbar kommentiert oder dokumentiert?
 - **Hat der Autor eindeutige und nachvollziehbare Namen für Variablen, Klassen und Methoden gewählt, die den Konventionen folgen?**
 - Sind die Kommentare des Autors sinnvoll und nachvollziehbar?
- Fehleranfälligkeit:
 - Werden Exceptions behandelt?
 - Existieren zu dem Code Tests? Sind die gängigen Testfälle abgedeckt?
 - Falls es eine Vorgabe zur Test-Abdeckung gibt, ist diese erfüllt?
- Performanz und Sicherheit:
 - Verbraucht der Code unnötige Ressourcen? (z.B. durch überflüssige Schleifen oder Datenbankabfragen, ineffizientes Dateimanagement, ...)
 - **Werden Usereingaben validiert und gesäubert, um unerwünschtes Verhalten, Code Injections etc. zu vermeiden?**
 - Liegen Keys, Passwörter etc. nicht als Hard Code vor?
- Style: Folgt der Code vorgegebenen Style-Guidelines?
- Dokumentation: Falls vorhanden, hat der Autor die Dokumentation aktualisiert?

Code-Prinzipien

Ein Code Reviewer sollte beim Review folgende Prinzipien berücksichtigen:

- Der neue Code soll die Qualität der Code-Basis nicht verschlechtern, muss sie aber nicht verbessern
- Es gibt keinen perfekten Code, nur besseren Code: Auch wenn man als Reviewer mit einer Umsetzung nicht hundertprozentig zufrieden ist, sollte der Code akzeptiert werden, solange er der Qualität der Code-Basis entspricht oder diese verbessert.

- Software Design ist selten eine Frage der persönlichen Referenz: Es gibt fast immer einen technischen Grund für Design-Entscheidungen. Technische Gründe und Fakten sind persönlichen Vorlieben immer vorzuziehen.

2. Wie gehen Sie vor?

Bei einem klassischen Code Review werden nur Teile eines Codes reviewt. Das weicht von Ihrem Code Review ab: Sie werden die gesamte Code Basis Ihres Kollegen reviewen. Bitte folgen Sie daher ungefähr diesem Ablauf:

Phase 1 - Zusammenfassung

Lassen Sie sich **für 5 bis 15 Minuten** (je nach Größe) das gesamte Projekt zusammenfassen:

- falls möglich, eine kurze Demo des fertigen Projektes, ansonsten eine Zusammenfassung des aktuellen Stands des Projekts
- Vorstellung des groben Aufbaus des Programms (gewählte Software Architektur, größere Code-Komponenten)
- Aufzeigen von eventuellen externen Schnittstellen (Datenbank, API, ...)

Halten Sie diesen Abschnitt bewusst kurz.

Phase 2 - Unterteilung in Abschnitte

Unterteilen Sie als Reviewer den Code gemeinsam mit dem Autor in sinnvolle Abschnitte, die sich einzeln reviewen lassen.

Verkünsteln Sie sich bei der Einteilung nicht. Bedenken Sie:

- Eine völlige Unabhängigkeit der Code Review-Abschnitte werden Sie nie erreichen können. Es wird immer Überlappungen geben.
- Es wird zeitlich nicht möglich sein, den gesamten Code zu reviewen. **Konzentrieren Sie sich auf wichtige Komponenten**, die:
 - besonders viel Programmlogik beinhalten (wie Controller)
 - kritische Sicherheitslücken enthalten können (wie externe Schnittstellen, Datenbankverbindungen)
 - grundlegende Software Design-Entscheidungen betreffen (wie Datenmodelle)
 - Tests durchführen (Unit und Integration Tests)

Bei einem kleineren Projekt mit MVC Pattern wären die drei Komponenten Model, Controller und View z.B. gut trennbare Komponenten, die sich einigermaßen eigenständig reviewen lassen.

Phase 3 - Code Review

Führen Sie für die einzelnen Komponenten ein In-Person-Code Review durch (siehe oben):

- Lesen Sie sich den Code kritisch durch und orientieren Sie sich dabei an den Code Review-Kriterien, insbesondere den fett gedruckten Fragen
- Stellen Sie dem Autor Fragen zum Code. Dies müssen nicht unbedingt offene Fragen sein, Sie können auch sein eigenes Code-Verständnis an dieser Stelle prüfen

- Führen Sie das Code Review hin zu möglichen Verbesserungen seinerseits, vermeiden Sie den "Ich habe es so gelöst"-Ansatz

Als Autor machen Sie sich Notizen zu den Verbesserungsvorschlägen oder setzen diese bei sehr kleinen Änderungen (wie Umbenennungen) direkt um.

Das Code Review einer einzelnen Komponente sollte, je nach Umfang und Komplexität, ca. 10 bis 30 Minuten in Anspruch nehmen.

Wiederholen Sie das Vorgehen für die einzelnen Komponenten. Ein zusammenhängendes Code Review sollte **nicht mehr als 1,5h bis 2h** umfassen - brechen Sie rechtzeitig ab und führen Sie bei Bedarf das Code Review zu einem anderen Zeitpunkt fort.

Phase 4 - Reflexion und Umsetzung der Verbesserungen

Reflektieren Sie zum Abschluss:

- Als Autor:
 - Was sollte ich an meinem eigenen Code verbessern?
 - Welche Erkenntnisse kann ich für das nächste Projekt mitnehmen?
- Als Reviewer:
 - Was habe ich beim Autor gesehen, das ich gut fand oder übernehmen könnte?

Falls Zeit besteht, nehmen Sie sich als Autor nach dem Code Review Zeit, die besprochenen Verbesserungen umzusetzen. Vereinbaren Sie einen Folgetermin, an dem Sie das Code Review fortsetzen.

3. Ergebnis des Code Reviews

Das konkrete Ergebnis des Code Reviews sollte für Sie als Autor sein:

- eine Liste mit Verbesserungsmöglichkeiten für Ihren Code
- falls die Zeit dafür besteht: verbesserter Code, den Sie erneut mit dem Reviewer besprechen können

Das didaktische Ergebnis sollte sein:

- Sie sind in der Lage, Ihren Code zu erklären und Design-Entscheidungen zu verteidigen
- Sie lernen für das nächste Projekt, welche Aspekte Sie mehr bedenken oder berücksichtigen sollten
- Sie verbessern Ihre Fähigkeit, sich in fremden Code einzulesen und diesen nachzuvollziehen
- Sie lesen Code nicht nur, sondern lernen, diesen kritisch zu hinterfragen